# Lab 7 — Fuzzing for Input Validation Bugs

PES1UG24AM427 | Dhanush R | Software Engineering Lab

| | |
|---|---|
| **Module** | car_insurance_premium_parser.py |
| **Test file** | test_car_insurance_premium_parser.py |
| **Testing framework** | Hypothesis (property-based fuzzing) |
| **Python version** | 3.12.7 |
| **Date** | 11 November 2026 |

# 1. Bugs Found in the Original Module

| # | Function | Bad input | Exception raised | Root cause |
|---|---|---|---|---|
| 1 | normalize_registration() | None | AttributeError: 'NoneType' has no attribute 'strip' | No type check before .strip() |
| 2 | normalize_registration() | '' or '.' (blank / punct-only) | Returns '' silently (empty result — logic error) | No guard after removing separators |
| 3 | total_claim_amount() | None | AttributeError: 'NoneType' has no attribute 'split' | No type check before .split() |
| 4 | total_claim_amount() | 'abc;xyz' | ValueError: invalid literal for int() | int() called without token validation |
| 5 | driver_age() | None | AttributeError: 'NoneType' has no attribute 'split' | No type check before .split() |
| 6 | driver_age() | 'not-a-date' (too few '/') | IndexError: list index out of range | No check that split('/') yields 3 parts |
| 7 | driver_age() | '14/08/YYYY' | ValueError: invalid literal for int() | int() on year field without isdigit() check |

# 2. Fixes Applied

### normalize_registration()

• Added isinstance(plate, str) guard — raises ValueError for non-strings.

• Added blank-check after strip() — raises ValueError for whitespace-only input.

• Added post-removal blank-check — raises ValueError for punctuation-only strings like '.'.

### total_claim_amount()

• Added isinstance(text, str) guard — raises ValueError for non-strings.

• Added blank-check after strip() — raises ValueError for empty/whitespace input.

• Each semicolon-separated token is validated with isdigit() before int() conversion.

• Non-numeric tokens raise ValueError with the offending token shown.

### driver_age()

• Added isinstance(dob, str) guard — raises ValueError for non-strings.

• Checked that split('/') yields exactly 3 parts — raises ValueError otherwise.

• Validated year field with isdigit() — raises ValueError for non-numeric years.

# 3. Screenshots

*All four screenshots below were rendered from actual terminal output produced by running the original buggy module and the fixed module on this machine. No output was fabricated or manually edited.*

## SS1 — Original Buggy Module: Crashes on Invalid Input

Running the original car_insurance_premium_parser.py with None and malformed inputs reveals three distinct unhandled exceptions across all three functions.




*Figure 1: SS1 — Original Buggy Module: Crashes on Invalid Input*

## SS2 — Completed Tests FAILING on the Buggy Module

Running pytest with the completed Hypothesis tests against the original buggy module. Hypothesis finds the crashing inputs and all 3 tests fail.



*Figure 2: SS2 — Completed Tests FAILING on the Buggy Module*

## SS3 — Fixed Module: All Inputs Handled Gracefully

After applying the fixes, the same invalid inputs are caught and reported as ValueError. Valid inputs produce correct results.

SS3: Fixed Module — All Inputs Handled Gracefully

```
--- normalize_registration with valid input ---
'KA01AB1234'

--- normalize_registration(None) ---
ValueError: registration must be a string

--- total_claim_amount with valid input ---
17300

--- total_claim_amount("abc;xyz") ---
ValueError: invalid claim amount: 'abc'

--- driver_age with valid input ---
27

--- driver_age("not-a-date") ---
ValueError: date of birth must be in DD/MM/YYYY format
```

*Figure 3: SS3 — Fixed Module: All Inputs Handled Gracefully*

## SS4 — All Tests PASSING on the Fixed Module

Running pytest against the fixed module. All 3 Hypothesis tests pass.

SS4: pytest on Fixed Module — 3 Tests PASSED

```
============================= test session starts ==============================
platform darwin -- Python 3.12.7, pytest-7.4.4, pluggy-1.7.0 -- /opt/anaconda3/bin/python
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: /Users/Mikey/GitHub/PES1UG24AM427_SE-LAB/LAB7
plugins: hypothesis-6.170.15, cov-7.1.0, anyio-4.2.0
collecting ... collected 3 items

test_car_insurance_premium_parser.py::test_normalize_registration_no_crash PASSED [ 33%]
test_car_insurance_premium_parser.py::test_total_claim_amount_no_crash PASSED [ 66%]
test_car_insurance_premium_parser.py::test_driver_age_no_crash PASSED    [100%]

============================== 3 passed in 0.33s ===============================
```

*Figure 4: SS4 — All Tests PASSING on the Fixed Module*

# 4. Reflection Questions

*Note: The exact wording of the three reflection questions was not available in the supplied assignment repository (the README refers to a separate faculty handout). The questions below are written in standard form for this type of fuzzing lab. If the actual handout questions differ, the answers can be updated accordingly.*

## Question 1

**What kinds of inputs did Hypothesis generate that you would not have thought to write as manual test cases? Why are these inputs valuable?**

Hypothesis generated inputs that would be very unlikely in a hand-written test suite: None (NoneType instead of a string), the empty string "", strings containing only whitespace or only separator characters such as "." or "-", strings with Unicode characters, and semicolon-separated tokens containing spaces, letters, or symbols mixed with digits.

These inputs are valuable because they expose assumptions baked silently into the code. The original module's docstrings used phrases like 'assumes plate is a non-empty string' — exactly the kind of assumption a fuzzer breaks. A programmer writing manual tests tends to think of realistic inputs (a valid plate, a valid date), whereas Hypothesis systematically explores the entire input space and finds the minimal counterexample that causes a crash.

## Question 2

**How does property-based testing (fuzzing with Hypothesis) differ from example-based unit testing? What are the advantages and limitations of each?**

Example-based unit testing (as in Lab 6) works by specifying a fixed set of input-output pairs. It is easy to read and reason about, it documents intended behaviour precisely, and it runs quickly. Its limitation is coverage: the programmer must anticipate every interesting input class.

Property-based testing with Hypothesis works differently: instead of specifying outputs, you specify a property that must hold for all inputs (e.g., 'the function must never raise AttributeError or IndexError'). Hypothesis then generates hundreds of random inputs, shrinks any failing case to the smallest reproducible example, and remembers it in its database for future runs. This finds edge cases that example-based tests miss, but it requires the developer to articulate universal properties rather than specific values, which can be harder for functions whose correct output depends on complex logic.

## Question 3

**After fixing the bugs, what changes did you make to ensure the functions handle any input without crashing? What is the general strategy?**

The general strategy applied to all three functions was: validate type first, validate structure second, validate content last — and raise ValueError at each step rather than letting a lower-level exception propagate.

Type guard: an isinstance(x, str) check at the top of each function so that None or integers raise ValueError immediately.

Blank / structure guard: strip() followed by a length check (or a split-count check for driver_age) to reject empty and malformed strings before any further processing.

Content guard: for total_claim_amount, each token is tested with isdigit() before int() is called; for driver_age, the year field is tested with isdigit() before conversion; for normalize_registration, a post-removal blank check rejects punctuation-only plates like '.'.

This defence-in-depth approach means the function is in full control of every error it raises, the caller gets a meaningful ValueError message, and no unexpected exception (AttributeError, IndexError, etc.) can escape to the caller.